1. Delete and earn

Solution:

```
class Solution {
public int deleteAndEarn(int[] nums) {
    int max = 0;

    for (int num : nums) {
        max = Math.max(max, num);
    }

    int[] freq = new int[max + 1];

    for (int num : nums) {
        freq[num]++;
    }

    int[] dp = new int[max + 1];

    dp[0] = 0;

    if (max >= 1) {
        dp[1] = freq[1];
    }

    for (int i = 2; i <= max; i++) {
        int take = dp[i - 2] + i * freq[i];
        int skip = dp[i - 1];

        dp[i] = Math.max(take, skip);
    }

    return dp[max];
  }
}
```

2. Partition-equal-subset-sum

Solution :

```java
class Solution {
    public boolean canPartition(int[] nums) {
        int n = nums.length;

        int total = 0;
        for(int num : nums){
            total += num;
        }

        if(total % 2 != 0){
            return false;
        }

        int half = total / 2;

        Boolean[][] dp = new Boolean[n + 1][half + 1];


        return helper(0, 0, n, nums, total, half, dp);
    }

    boolean helper(int index, int sum, int n, int[]
nums, int total, int half, Boolean[][] dp){

        if(sum == half){
            return true;
        }

        if (sum > half) {
            return false;
        }

        if(index == n){
            return false;
        }

        if(dp[index][sum] != null){
            return dp[index][sum];
        }


        boolean notTake = helper(index + 1, sum, n,
nums, total, half, dp);
        boolean take = helper(index + 1, sum +
nums[index], n, nums, total, half, dp);

        return dp[index][sum] = take || notTake;
    }
}
```

3. Target-sum

Solution :

```
class Solution {
    public int findTargetSumWays(int[] nums, int target)
{
        int n = nums.length;

        int sum = 0;
        int[][] dp = new int[][2 * sum + 1];
        Arrays.fill(dp, -1);

        for(int i=0; i<n; i++){
            sum += nums[i];
        }

        return helper(0, 0, nums, target, 0, dp);

    }

    int helper(int index,int sum, int[] nums, int
target, int count, int[] dp){

        if(index == nums.length){
            if(sum == target){
                return 1;
            }
            return 0;
        }

        if(dp[index][s] != -1){
            return dp[index];
        }

        int add =  helper(index + 1 ,sum + nums[index],
nums, target, count, dp);

        int subtract = helper(index + 1, sum -
nums[index], nums, target, count, dp);

        return dp[index] = add + subtract;
    }
}
```

4. Minimum-cost-for-tickets

Solution :

```
        class Solution {
    public int mincostTickets(int[] days, int[] costs) {

        int n = days.length;

        int[] dp = new int[n + 1];

        for (int i = n - 1; i >= 0; i--) {

            int j = i;

            while (j < n && days[j] < days[i] + 1) {
                j++;
            }

            int oneDay = costs[0] + dp[j];


            j = i;

            while (j < n && days[j] < days[i] + 7) {
                j++;
            }

            int sevenDay = costs[1] + dp[j];

            j = i;

            while (j < n && days[j] < days[i] + 30) {
                j++;
            }

            int thirtyDay = costs[2] + dp[j];

            dp[i] = Math.min(oneDay,
                    Math.min(sevenDay, thirtyDay));
        }

        return dp[0];
    }
}
```

5. Last stone weights-2

Solution :

```
class Solution {
    public int stoneWeight(int[] stones) {

        int total = 0;

        for (int stone : stones) {
            total += stone;
        }

        int target = total / 2;

        boolean[][] dp =
            new boolean[stones.length + 1][target + 1];

        dp[0][0] = true;

        for (int i = 1; i <= stones.length; i++) {

            for (int j = 0; j <= target; j++) {

                dp[i][j] = dp[i - 1][j];


                if (j >= stones[i - 1]) {
                    dp[i][j] = dp[i][j] ||
                            dp[i - 1][j - stones[i - 1]];
                }
            }
        }

        int best = 0;

        for (int j = target; j >= 0; j--) {
            if (dp[stones.length][j]) {
                best = j;
                break;
            }
        }

        return total - 2 * best;
    }
}
```

6. Problemset

Solution :

```
import java.util.*;

public class Main {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      int[] freq = new int[100001];

      for (int i = 0; i < n; i++) {
         int x = sc.nextInt();
         freq[x]++;
      }

      long[] dp = new long[100001];

      dp[0] = 0;
      dp[1] = freq[1];

      for (int i = 2; i <= 100000; i++) {

         long take = dp[i - 2] + (long) i * freq[i];

         long skip = dp[i - 1];

         dp[i] = Math.max(take, skip);
      }

      System.out.println(dp[100000]);
   }
}
```

7. Problemset - 1195C

Solution :

```
public class Main {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      long[][] a = new long[2][n];

      for (int i = 0; i < n; i++) {
         a[0][i] = sc.nextLong();
      }

      for (int i = 0; i < n; i++) {
```

```
            a[1][i] = sc.nextLong();
        }

        long[][] dp = new long[2][n];

        dp[0][0] = a[0][0];
        dp[1][0] = a[1][0];

        if (n >= 2) {
            dp[0][1] = Math.max(a[0][0], a[0][1]);
            dp[1][1] = Math.max(a[1][0], a[1][1]);
        }

        for (int i = 2; i < n; i++) {

            dp[0][i] = Math.max(
                dp[0][i - 1],
                Math.max(
                    dp[1][i - 1],
                    dp[1][i - 2]
                ) + a[0][i]
            );

            dp[1][i] = Math.max(
                dp[1][i - 1],
                Math.max(
                    dp[0][i - 1],
                    dp[0][i - 2]
                ) + a[1][i]
            );
        }

        System.out.println(
            Math.max(dp[0][n - 1], dp[1][n - 1])
        );
    }
}
```

8. 1097B

Solution :

```
public class Main {

    static int n;
    static int[] a;
    static boolean found = false;

    static void solve(int index, int angle) {

        if (index == n) {
            if (angle % 360 == 0) {
                found = true;
```

```
            }
            return;
        }

        if (found) {
            return;
        }

        solve(index + 1, angle + a[index]);

        solve(index + 1, angle - a[index]);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        n = sc.nextInt();

        a = new int[n];

        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }

        solve(0, 0);

        if (found) {
            System.out.println("YES");
        } else {
            System.out.println("NO");
        }
    }
}
```

9. 698 A

Solution :

```
public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] a = new int[n];

        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }

        int[][] dp = new int[n][3];

        dp[0][0] = 1;
        dp[0][1] = 1000;
```

```
        dp[0][2] = 1000;

        if (a[0] == 1 || a[0] == 3) {
            dp[0][1] = 0;
        }

        if (a[0] == 2 || a[0] == 3) {
            dp[0][2] = 0;
        }

        for (int i = 1; i < n; i++) {

            dp[i][0] =
                Math.min(
                    dp[i - 1][0],
                    Math.min(dp[i - 1][1], dp[i - 1][2])
                ) + 1;


            dp[i][1] = 1000;

            if (a[i] == 1 || a[i] == 3) {
                dp[i][1] =
                    Math.min(dp[i - 1][0], dp[i - 1][2]);
            }

            dp[i][2] = 1000;

            if (a[i] == 2 || a[i] == 3) {
                dp[i][2] =
                    Math.min(dp[i - 1][0], dp[i - 1][1]);
            }
        }

        System.out.println(
            Math.min(
                dp[n - 1][0],
                Math.min(dp[n - 1][1], dp[n - 1][2])
            )
        );
    }
}
```

10. 1418 C

Solution :

```
public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();
```

```
while (t-- > 0) {

   int n = sc.nextInt();

   int[] a = new int[n];

   for (int i = 0; i < n; i++) {
      a[i] = sc.nextInt();
   }



   int[][] dp = new int[n + 1][2];

   for (int i = 0; i <= n; i++) {
      dp[i][0] = 1000000;
      dp[i][1] = 1000000;
   }

   dp[0][0] = 0;

   for (int i = 0; i < n; i++) {


      if (dp[i][0] != 1000000) {


         dp[i + 1][1] = Math.min(
            dp[i + 1][1],
            dp[i][0] + a[i]
         );


         if (i + 1 < n) {
            dp[i + 2][1] = Math.min(
               dp[i + 2][1],
               dp[i][0] + a[i] + a[i + 1]
            );
         }
      }

      if (dp[i][1] != 1000000) {


         dp[i + 1][0] = Math.min(
            dp[i + 1][0],
            dp[i][1]
         );


         if (i + 1 < n) {
            dp[i + 2][0] = Math.min(
               dp[i + 2][0],
               dp[i][1]
            );
         }
      }
   }
```

```
            System.out.println(
                Math.min(dp[n][0], dp[n][1])
            );
        }
    }
}
```